DESARROLLO WEB CLIENTE - 2º DAW

UD 02: Javascript y React

MANUAL DETALLADO DE CONFIGURACIÓN Y EJECUCIÓN

EJEMPLOS DE AUTOAPRENDIZAJE EA25 - EA30

Curso 2026-2027

Elaborado por: José Fernández González

Versión 1.0 · octubre de 2026

CC BY-NC-ND 4.0

https://github.com/jstechg/material-docente/tree/main/2DAW_Desarrollo_Web_Cliente

Indicaciones previas

Este manual explica cómo configurar, ejecutar y comprobar los ejemplos de React EA25 a EA30 en la máquina virtual Ubuntu preparada para DWEC. La estrategia de carpetas se ha modificado respecto a los enunciados originales: cada ejemplo se realiza en un proyecto Vite independiente y no se reutilizan los ficheros del ejemplo anterior.

Nota: Se utiliza como carpeta base ~/DWEC/EA02. El símbolo ~ representa la carpeta personal del usuario. Esta forma evita trabajar directamente bajo /DWEC, que normalmente requeriría permisos de administrador.

Estructura final de carpetas

```
~/DWEC/EA02/
├── EA25/
├── EA26/
├── EA27/
├── EA28/
├── EA29/
└── EA30/
```

Cada carpeta EAxx contiene su propio package.json, node_modules, configuración de ESLint, src y resto de ficheros generados por Vite. Por tanto, los seis ejemplos pueden abrirse, modificarse y probarse de forma totalmente independiente.

Comprobación inicial del entorno

Antes de comenzar EA25, comprueba una vez que Node.js, npm, Vite y Visual Studio Code están disponibles:

```
node --version
npm --version
vite --version
code --version | head -1
```

También es recomendable comprobar que Firefox está instalado:

```
firefox --version
```

Mapa de los seis ejemplos

Ejemplo	Contenido principal	Ficheros propios	Prueba específica
EA25	Proyecto Vite + primer componente	App.jsx, index.css	Página básica y lint
EA26	props y desestructuración	TarjetaModulo.jsx, App.jsx	Cambiar props
EA27	children y composición	Panel.jsx, App.jsx	Añadir un tercer Panel
EA28	JSX, fragmentos, className y ternarios	EstadoEntrega.jsx, App.jsx, index.css	Probar 20, 60 y 100
EA29	map(), key y CSS por componente	ListaRecursos.jsx/.css, App.jsx	Añadir un cuarto objeto
EA30	Integración de componentes	Persona.jsx, FichaCurso.jsx/.css, App.jsx	Tercer participante y array vacío

EA25. React I: crear el proyecto y primer componente

EA25 es el ejemplo base. Aquí se describe el proceso completo que se reutilizará como referencia en EA26-EA30: crear un proyecto Vite + React independiente, instalar dependencias, abrirlo en VS Code, copiar el código, comprobar ESLint, ejecutar Vite y validar el resultado en Firefox.

EA25.1. Crear un proyecto Vite + React independiente

Cada ejemplo se realiza en un proyecto independiente. Para EA25, sitúate en la carpeta base y crea primero un proyecto con nombre temporal en minúsculas. Después renombra la carpeta a EA25. Se usa el nombre temporal en minúsculas para que el nombre del paquete npm sea válido.

```
mkdir -p ~/DWEC/EA02
cd ~/DWEC/EA02
npm create vite@latest ea25
```

En el asistente de Vite selecciona, en este orden:

- React
- JavaScript
- ESLint
- En "Install with npm and start now?", selecciona No.

Cuando termine el asistente, renombra la carpeta y entra en ella:

```
mv ea25 EA25
cd EA25
```

Instala las dependencias del proyecto:

```
npm install
```

Comprueba la carpeta actual y que existen package.json y src:

```
pwd
ls -la
ls -la src
```

Abre el proyecto completo con Visual Studio Code:

```
code .
```

Nota: Si Vite arranca el servidor automáticamente pese a haber elegido No, deténlo con Ctrl+C antes de continuar.

EA25.2. Sustituir los ficheros del ejemplo

En VS Code abre src/App.jsx, elimina su contenido y sustitúyelo por el siguiente:

src/App.jsx

```
function App() {
  return (
    <main>
      <h1>DWEC - UT02</h1>
      <p>Primer componente funcional.</p>
      <p>El JSX se devuelve desde return.</p>
    </main>
  );
}

export default App;
```

A continuación abre src/index.css y sustitúyelo por:

src/index.css

```
body {
  font-family: sans-serif;
  margin: 2rem;
}
```

Nota: Mantén src/main.jsx e index.html tal como los ha generado Vite. No necesitas modificarlos para este ejemplo.

EA25.3. Comprobar la estructura

```
find src -maxdepth 2 -type f | sort
```

Como mínimo deben aparecer src/App.jsx, src/index.css y src/main.jsx.

EA25.4. Qué debes observar en el código

- App es un componente funcional.
- El componente devuelve JSX desde return.
- El nombre del componente comienza por mayúscula.
- El JSX utiliza un único elemento raíz <main>.

EA25.5. Comprobar el código con ESLint

Desde el terminal situado en la raíz del proyecto ejecuta:

```
npm run lint
```

El comando debe terminar sin errores. Opcionalmente puedes comprobar el código de retorno:

```
echo $?
```

Un valor 0 indica que el último comando terminó correctamente.

EA25.6. Arrancar Vite y abrir la aplicación

```
npm run dev
```

Vite mostrará una URL local. Normalmente será http://localhost:5173/, pero si ese puerto está ocupado utilizará otro. Mantén este terminal abierto mientras pruebas la aplicación.

En otro terminal puedes abrir Firefox directamente:

```
firefox http://localhost:5173
```

Nota: Si Vite muestra un puerto diferente, utiliza exactamente la URL indicada en la línea "Local:".

EA25.7. Comprobaciones en el navegador

Pulsa F12 en Firefox y revisa la pestaña Consola. No deben aparecer errores de JavaScript o React relacionados con el ejemplo. También puedes usar Inspector para observar el HTML que React ha generado a partir del JSX.

Resultado esperado / comprobaciones

- La página muestra el título «DWEC - UT02».
- Aparecen los dos párrafos del ejemplo.
- npm run lint termina sin errores.

EA25.8. Prueba de actualización automática

Con npm run dev activo, cambia temporalmente el título de App.jsx a «DWEC - UT02 - Prueba», guarda con Ctrl+S y observa que Vite actualiza la página. Después devuelve el texto a su valor original.

EA25.9. Comprobación final de construcción

Cuando hayas terminado las pruebas, vuelve al terminal donde se ejecuta Vite y pulsa Ctrl+C. A continuación genera la versión de producción:

```
npm run build
ls -la dist
```

Si la construcción termina sin errores y existe dist/, puedes probarla con:

```
npm run preview
```

Vite mostrará normalmente http://localhost:4173/. Ábrela en Firefox y, al terminar, detén el servidor con Ctrl+C.

EA26. React II: props y desestructuración

EA26 se realiza en un proyecto nuevo. No reutilices la carpeta EA25. Repite el procedimiento de creación descrito en EA25.1, sustituyendo ea25/EA25 por ea26/EA26. Los comandos concretos son los siguientes:

```
cd ~/DWEC/EA02
npm create vite@latest ea26
# Selecciona React -> JavaScript -> ESLint -> No
mv ea26 EA26
cd EA26
npm install
mkdir -p src/components
code .
```

Nota: A partir de aquí, la diferencia con EA25 está en los ficheros del ejemplo. EA26 introduce un componente hijo reutilizable y el paso de datos mediante props.

EA26.2. Crear TarjetaModulo.jsx

src/components/TarjetaModulo.jsx

```
function TarjetaModulo({ nombre, curso, horas }) {
  return (
    <article>
      <h2>{nombre}</h2>
      <p>Curso: {curso}</p>
      <p>Horas semanales: {horas}</p>
    </article>
  );
}

export default TarjetaModulo;
```

EA26.3. Sustituir App.jsx

src/App.jsx

```
import TarjetaModulo from "./components/TarjetaModulo.jsx";

function App() {
  return (
    <main>
      <h1>Módulos</h1>
      <TarjetaModulo nombre="DWEC" curso="2.º DAW" horas={3} />
      <TarjetaModulo nombre="Sistemas Informáticos" curso="1.º DAW" horas={5} />
    </main>
  );
}

export default App;
```

No es necesario modificar index.css, main.jsx ni index.html para realizar el ejemplo.

EA26.4. Comprobar la estructura

```
find src -maxdepth 2 -type f | sort
```

Debe existir src/components/TarjetaModulo.jsx además de App.jsx, index.css y main.jsx.

EA26.5. Comprobar el código con ESLint

Desde el terminal situado en la raíz del proyecto ejecuta:

```
npm run lint
```

El comando debe terminar sin errores. Opcionalmente puedes comprobar el código de retorno:

```
echo $?
```

Un valor 0 indica que el último comando terminó correctamente.

EA26.6. Arrancar Vite y abrir la aplicación

```
npm run dev
```

Vite mostrará una URL local. Normalmente será http://localhost:5173/, pero si ese puerto está ocupado utilizará otro. Mantén este terminal abierto mientras pruebas la aplicación.

En otro terminal puedes abrir Firefox directamente:

```
firefox http://localhost:5173
```

Nota: Si Vite muestra un puerto diferente, utiliza exactamente la URL indicada en la línea "Local:".

EA26.7. Comprobaciones en el navegador

Pulsa F12 en Firefox y revisa la pestaña Consola. No deben aparecer errores de JavaScript o React relacionados con el ejemplo. También puedes usar Inspector para observar el HTML que React ha generado a partir del JSX.

Resultado esperado / comprobaciones

- Aparecen dos tarjetas creadas por el mismo componente TarjetaModulo.
- La primera muestra DWEC, 2.º DAW y 3 horas; la segunda Sistemas Informáticos, 1.º DAW y 5 horas.

EA26.8. Prueba específica: cambiar sólo las props

Mantén npm run dev activo. En App.jsx cambia únicamente las props de la segunda instancia, por ejemplo:

```
<TarjetaModulo nombre="Programación" curso="1.º DAM" horas={6} />
```

Guarda y comprueba que cambia la segunda tarjeta sin modificar TarjetaModulo.jsx. Después devuelve App.jsx al contenido original.

Nota: No modifiques las props dentro de TarjetaModulo. Si necesitas transformar un valor, crea una variable local derivada.

EA26.9. Comprobación final de construcción

Cuando hayas terminado las pruebas, vuelve al terminal donde se ejecuta Vite y pulsa Ctrl+C. A continuación genera la versión de producción:

```
npm run build
ls -la dist
```

Si la construcción termina sin errores y existe dist/, puedes probarla con:

```
npm run preview
```

Vite mostrará normalmente http://localhost:4173/. Ábrela en Firefox y, al terminar, detén el servidor con Ctrl+C.

EA27. React III: children y composición

EA27 vuelve a comenzar desde un proyecto Vite independiente. Repite EA25.1 cambiando ea25/EA25 por ea27/EA27. Después crea la carpeta de componentes:

```
cd ~/DWEC/EA02
npm create vite@latest ea27
# Selecciona React -> JavaScript -> ESLint -> No
mv ea27 EA27
cd EA27
npm install
mkdir -p src/components
code .
```

EA27.2. Crear Panel.jsx

src/components/Panel.jsx

```
function Panel({ titulo, children }) {
  return (
    <section>
      <h2>{titulo}</h2>
      <div>{children}</div>
    </section>
  );
}

export default Panel;
```

EA27.3. Sustituir App.jsx

src/App.jsx

```
import Panel from "./components/Panel.jsx";

function App() {
  return (
    <main>
      <Panel titulo="Recordatorio">
        <p>Usa nombres lowerCamelCase en JavaScript.</p>
      </Panel>

      <Panel titulo="Objetivo">
        <p>Separar la interfaz en piezas reutilizables.</p>
        <strong>Este contenido llega mediante children.</strong>
      </Panel>
    </main>
  );
}

export default App;
```

No modifiques index.css, main.jsx ni index.html.

EA27.4. Qué debes observar

- titulo llega como una prop normal.
- children contiene el JSX escrito entre <Panel> y </Panel>.
- La misma estructura Panel admite contenidos interiores diferentes.
- Panel separa la estructura visual del contenido que recibe.

EA27.5. Comprobar el código con ESLint

Desde el terminal situado en la raíz del proyecto ejecuta:

```
npm run lint
```

El comando debe terminar sin errores. Opcionalmente puedes comprobar el código de retorno:

```
echo $?
```

Un valor 0 indica que el último comando terminó correctamente.

EA27.6. Arrancar Vite y abrir la aplicación

```
npm run dev
```

Vite mostrará una URL local. Normalmente será http://localhost:5173/, pero si ese puerto está ocupado utilizará otro. Mantén este terminal abierto mientras pruebas la aplicación.

En otro terminal puedes abrir Firefox directamente:

```
firefox http://localhost:5173
```

Nota: Si Vite muestra un puerto diferente, utiliza exactamente la URL indicada en la línea "Local:".

EA27.7. Comprobaciones en el navegador

Pulsa F12 en Firefox y revisa la pestaña Consola. No deben aparecer errores de JavaScript o React relacionados con el ejemplo. También puedes usar Inspector para observar el HTML que React ha generado a partir del JSX.

Resultado esperado / comprobaciones

- Aparece un panel «Recordatorio» con un párrafo.
- Aparece un panel «Objetivo» con un párrafo y un elemento strong.
- Ambos reutilizan exactamente el mismo componente Panel.

EA27.8. Prueba específica: añadir un tercer Panel

Con el servidor activo, añade dentro de <main> un tercer Panel que utilice una lista como children:

```
<Panel titulo="Tecnologías">
  <ul>
    <li>JavaScript</li>
    <li>React</li>
    <li>Vite</li>
  </ul>
</Panel>
```

Guarda y comprueba que el tercer panel aparece sin cambiar Panel.jsx. Después ejecuta de nuevo npm run lint.

EA27.9. Comprobación final de construcción

Cuando hayas terminado las pruebas, vuelve al terminal donde se ejecuta Vite y pulsa Ctrl+C. A continuación genera la versión de producción:

```
npm run build
ls -la dist
```

Si la construcción termina sin errores y existe dist/, puedes probarla con:

```
npm run preview
```

Vite mostrará normalmente http://localhost:4173/. Ábrela en Firefox y, al terminar, detén el servidor con Ctrl+C.

EA28. React IV: JSX, fragmentos, className y renderizado condicional

Crea un proyecto independiente para EA28 repitiendo EA25.1 con los nombres ea28/EA28:

```
cd ~/DWEC/EA02
npm create vite@latest ea28
# Selecciona React -> JavaScript -> ESLint -> No
mv ea28 EA28
cd EA28
npm install
mkdir -p src/components
code .
```

EA28.2. Crear EstadoEntrega.jsx

src/components/EstadoEntrega.jsx

```
function EstadoEntrega({ completada, porcentaje }) {
  const texto = completada ? "Entrega completada" : "Entrega pendiente";

  return (
    <>
      <h2 className="estado-titulo">{texto}</h2>
      <p>Progreso: {porcentaje}%</p>
      <p>{porcentaje >= 50 ? "Buen avance" : "Conviene continuar"}</p>
    </>
  );
}

export default EstadoEntrega;
```

EA28.3. Sustituir App.jsx e index.css

src/App.jsx

```
import EstadoEntrega from "./components/EstadoEntrega.jsx";

function App() {
  return (
    <main>
      <EstadoEntrega completada={false} porcentaje={40} />
      <hr />
      <EstadoEntrega completada={true} porcentaje={100} />
    </main>
  );
}

export default App;
```

src/index.css

```
body {
  font-family: sans-serif;
  margin: 2rem;
}

.estado-titulo {
  border-bottom: 1px solid #777;
}
```

EA28.4. Qué debes observar

- Las expresiones JavaScript se insertan en JSX entre llaves { }.
- Los dos mensajes condicionales se resuelven con el operador ternario.
- className se utiliza en JSX en lugar del atributo HTML class.
- El fragmento <>...</> agrupa varios elementos hermanos sin añadir un nodo HTML adicional.

EA28.5. Comprobar el código con ESLint

Desde el terminal situado en la raíz del proyecto ejecuta:

```
npm run lint
```

El comando debe terminar sin errores. Opcionalmente puedes comprobar el código de retorno:

```
echo $?
```

Un valor 0 indica que el último comando terminó correctamente.

EA28.6. Arrancar Vite y abrir la aplicación

```
npm run dev
```

Vite mostrará una URL local. Normalmente será http://localhost:5173/, pero si ese puerto está ocupado utilizará otro. Mantén este terminal abierto mientras pruebas la aplicación.

En otro terminal puedes abrir Firefox directamente:

```
firefox http://localhost:5173
```

Nota: Si Vite muestra un puerto diferente, utiliza exactamente la URL indicada en la línea "Local:".

EA28.7. Comprobaciones en el navegador

Pulsa F12 en Firefox y revisa la pestaña Consola. No deben aparecer errores de JavaScript o React relacionados con el ejemplo. También puedes usar Inspector para observar el HTML que React ha generado a partir del JSX.

Resultado esperado / comprobaciones

- Para completada={false} y porcentaje={40}: «Entrega pendiente» y «Conviene continuar».
- Para completada={true} y porcentaje={100}: «Entrega completada» y «Buen avance».
- El título utiliza la clase CSS estado-titulo.

EA28.8. Prueba específica: porcentajes 20, 60 y 100

Mantén npm run dev activo y modifica temporalmente la prop porcentaje de una instancia de EstadoEntrega. Prueba sucesivamente:

```
porcentaje={20}
porcentaje={60}
porcentaje={100}
```

Con 20 debe aparecer «Conviene continuar»; con 60 y 100 debe aparecer «Buen avance». Como prueba de frontera, puedes comprobar también porcentaje={50}, que debe producir «Buen avance» porque la condición utiliza >= 50.

Nota: En el Inspector del navegador comprobarás que el fragmento <>...</> no genera una etiqueta HTML propia.

EA28.9. Comprobación final de construcción

Cuando hayas terminado las pruebas, vuelve al terminal donde se ejecuta Vite y pulsa Ctrl+C. A continuación genera la versión de producción:

```
npm run build
ls -la dist
```

Si la construcción termina sin errores y existe dist/, puedes probarla con:

```
npm run preview
```

Vite mostrará normalmente http://localhost:4173/. Ábrela en Firefox y, al terminar, detén el servidor con Ctrl+C.

EA29. React V: listas con map(), key y CSS por componente

Crea EA29 como un proyecto Vite independiente, exactamente igual que en EA25.1, cambiando los nombres del proyecto:

```
cd ~/DWEC/EA02
npm create vite@latest ea29
# Selecciona React -> JavaScript -> ESLint -> No
mv ea29 EA29
cd EA29
npm install
mkdir -p src/components
code .
```

EA29.2. Crear ListaRecursos.jsx

src/components/ListaRecursos.jsx

```
import "./ListaRecursos.css";

function ListaRecursos({ recursos }) {
  return (
    <section className="recursos">
      <h2>Recursos de estudio</h2>
      <ul>
        {recursos.map((recurso) => (
          <li key={recurso.id}>
            <strong>{recurso.codigo}</strong>: {recurso.nombre}
          </li>
        ))}
      </ul>
    </section>
  );
}

export default ListaRecursos;
```

EA29.3. Crear el CSS del componente

src/components/ListaRecursos.css

```
.recursos {
  border: 1px solid #777;
  padding: 1rem;
  max-width: 520px;
}

.recursos li {
  margin: 0.4rem 0;
}
```

EA29.4. Sustituir App.jsx

src/App.jsx

```
import ListaRecursos from "./components/ListaRecursos.jsx";

function App() {
  const recursos = [
    { id: 1, codigo: "JS", nombre: "Sintaxis" },
    { id: 2, codigo: "OBJ", nombre: "Objetos" },
    { id: 3, codigo: "REACT", nombre: "Componentes" },
  ];

  return (
    <main>
      <ListaRecursos recursos={recursos} />
    </main>
  );
}
```

```
export default App;
```

Observa que ListaRecursos.jsx importa su propio fichero CSS mediante import "./ListaRecursos.css";.

EA29.5. Qué debes observar

- App crea un array de objetos con id, codigo y nombre.
- ListaRecursos recibe el array mediante la prop recursos.
- map() transforma cada objeto en un elemento <li>.
- Cada <li> utiliza key={recurso.id}.
- El CSS queda asociado al componente mediante un fichero externo.

EA29.6. Comprobar con ESLint y ejecutar

```
npm run lint
npm run dev
```

Abre la URL que muestre Vite, normalmente con:

```
firefox http://localhost:5173
```

Pulsa F12 y comprueba que la consola no muestra errores.

Resultado esperado / comprobaciones

- Se generan tres elementos <li> a partir de los tres objetos del array.
- Cada elemento muestra su código y su nombre.

EA29.7. Prueba específica: añadir un cuarto objeto

Añade un cuarto objeto al array recursos de App.jsx, por ejemplo:

```
{ id: 4, codigo: "DOM", nombre: "Document Object Model" },
```

Guarda y comprueba que aparece automáticamente un cuarto <li> sin cambiar ListaRecursos.jsx.

Nota: Cuando existe un identificador estable como id, es mejor usarlo como key que el índice del array, porque identifica de forma estable cada elemento aunque la lista cambie de orden, inserte o elimine elementos.

EA29.8. Comprobación final de construcción

Cuando hayas terminado las pruebas, vuelve al terminal donde se ejecuta Vite y pulsa Ctrl+C. A continuación genera la versión de producción:

```
npm run build
ls -la dist
```

Si la construcción termina sin errores y existe dist/, puedes probarla con:

```
npm run preview
```

Vite mostrará normalmente http://localhost:4173/. Ábrela en Firefox y, al terminar, detén el servidor con Ctrl+C.

EA30. React VI: miniaplicación integrada por componentes

EA30 integra los conceptos anteriores. Crea un último proyecto Vite independiente:

```
cd ~/DWEC/EA02
npm create vite@latest ea30
# Selecciona React -> JavaScript -> ESLint -> No
mv ea30 EA30
cd EA30
npm install
mkdir -p src/components
code .
```

EA30.2. Crear Persona.jsx

src/components/Persona.jsx

```
function Persona({ nombre, rol, children }) {
  return (
    <article className="persona">
      <h3>{nombre}</h3>
      <p>Rol: {rol}</p>
      <div>{children}</div>
    </article>
  );
}

export default Persona;
```

EA30.3. Crear FichaCurso.jsx

src/components/FichaCurso.jsx

```
import Persona from "./Persona.jsx";
import "./FichaCurso.css";

function FichaCurso({ titulo, coordinador, participantes, children }) {
  return (
    <section className="ficha-curso">
      <h1>{titulo}</h1>
      <p>Coordinación: {coordinador}</p>
      <div className="descripcion">{children}</div>

      <h2>Participantes</h2>
      {participantes.length === 0 ? (
        <p>No hay participantes.</p>
      ) : (
        participantes.map((persona) => (
          <Persona key={persona.id} nombre={persona.nombre} rol={persona.rol}>
            <p>{persona.presentacion}</p>
          </Persona>
        ))
      )}
    </section>
  );
}

export default FichaCurso;
```

EA30.4. Crear FichaCurso.css

src/components/FichaCurso.css

```
.ficha-curso {
  max-width: 700px;
  margin: 0 auto;
  font-family: sans-serif;
}

.descripcion,
.persona {
```

```
  border: 1px solid #999;
  padding: 0.8rem;
  margin: 0.8rem 0;
}
```

EA30.5. Sustituir App.jsx

src/App.jsx

```
import FichaCurso from "./components/FichaCurso.jsx";

function App() {
  const participantes = [
    { id: 1, nombre: "Alicia", rol: "desarrollo", presentacion: "Trabaja con JavaScript." },
    { id: 2, nombre: "Bruno", rol: "diseño", presentacion: "Cuida la presentación visual." },
  ];

  return (
    <FichaCurso
      titulo="Taller de interfaces"
      coordinador="Equipo DWEC"
      participantes={participantes}
    >
      <p>Ejemplo de composición de componentes, props, children y listas.</p>
    </FichaCurso>
  );
}

export default App;
```

EA30.6. Qué debes observar

- App prepara los datos participantes y se los pasa a FichaCurso mediante props.
- FichaCurso recibe también children para la descripción.
- FichaCurso utiliza un ternario para distinguir entre una lista vacía y una lista con participantes.
- map() genera un componente Persona por cada objeto.
- Cada Persona recibe props y children.
- FichaCurso importa un CSS específico del componente.

EA30.7. Comprobar con ESLint

```
npm run lint
```

Corrige cualquier error antes de continuar. No arranques Vite mientras npm run lint siga mostrando errores.

EA30.8. Ejecutar y comprobar la ficha

```
npm run dev
```

Abre la URL local en Firefox, normalmente:

```
firefox http://localhost:5173
```

Pulsa F12 y revisa Consola e Inspector.

Resultado esperado / comprobaciones

- La ficha muestra «Taller de interfaces» y «Equipo DWEC».
- Aparece el contenido children de descripción.
- Se genera una tarjeta Persona para Alicia y otra para Bruno.

EA30.9. Prueba específica 1: añadir un tercer participante

Añade al array participantes un tercer objeto con un id diferente, por ejemplo:

```
{ id: 3, nombre: "Carla", rol: "pruebas", presentacion: "Comprueba el funcionamiento." },
```

Guarda y comprueba que React genera automáticamente una tercera tarjeta Persona.

EA30.10. Prueba específica 2: comprobar el ternario con array vacío

Después de la prueba anterior, sustituye temporalmente la prop en App.jsx por:

```
participantes={[]}
```

La lista de personas debe desaparecer y en su lugar debe mostrarse «No hay participantes». Después devuelve App.jsx a su contenido original y ejecuta otra vez npm run lint.

EA30.11. Comprobación final de construcción

Cuando hayas terminado las pruebas, vuelve al terminal donde se ejecuta Vite y pulsa Ctrl+C. A continuación genera la versión de producción:

```
npm run build
ls -la dist
```

Si la construcción termina sin errores y existe dist/, puedes probarla con:

```
npm run preview
```

Vite mostrará normalmente http://localhost:4173/. Ábrela en Firefox y, al terminar, detén el servidor con Ctrl+C.

Resumen rápido de ejecución

Una vez creado cada proyecto y copiados sus ficheros, la secuencia normal de trabajo dentro de cualquier carpeta EA25-EA30 es:

```
npm run lint
npm run dev
```

Abrir en otro terminal:

```
firefox http://localhost:5173
```

Al terminar:

```
# En el terminal de Vite: Ctrl+C
npm run build
npm run preview
```

Nota: Cada ejemplo es independiente. Para trabajar con otro EA, detén el servidor anterior, entra en la carpeta correspondiente y ejecuta allí npm run lint / npm run dev.

Estructura de referencia de cada proyecto

```
EAxx/
├── node_modules/
├── public/
├── src/
│   ├── components/      # cuando el ejemplo lo necesite
│   ├── App.jsx
│   ├── index.css
│   └── main.jsx
├── index.html
├── package.json
├── package-lock.json
└── ...
```